

아래 "붙여넣을 내용" 전체를 그대로 복사해서 AI B(ChatGPT 등, Claude가 아닌 다른 서비스)의 새 대화 맨 첫 메시지로 붙여넣으세요. 파일 업로드가 가능하면 `ex_5` 폴더를 zip으로 올려도 되지만, 못 올려도 아래 내용만으로 작업할 수 있도록 필요한 파일 전체를 이미 담아뒀습니다.

---붙여넣을 내용 시작---

나는 지금 "인수인계 문서만으로 대화 없이 작업을 이어받을 수 있는가"를 검증하는 실험을 하고 있어. 너는 이 프로젝트에 대한 어떤 이전 대화 기록도 받지 않았고, 받아서도 안 돼. 아래 인수인계 문서와 저장소 파일 내용이 네가 아는 전부야. 부족한 정보가 있으면 추측하지 말고 "문서 누락"이라고 명시해줘.

## 인수인계 문서

### 1. 목표

USD 기준 환율 정보판(정적 웹페이지, HTML/CSS/JS만 사용, 서버·빌드 없음)에 통화 선택 기능을 추가하는 작업이야. 사용자가 KRW/EUR/JPY/GBP 중 하나를 고르면, 현재값뿐 아니라 "날짜별 기록"과 "어제 대비 변화"도 그 통화 기준으로 정확히 동작해야 해. 지금은 "날짜별 기록/어제 대비 변화"가 KRW만 지원돼 — 이 부분을 완성하는 게 네(AI B) 목표야.

### 2. 현재 상태

- 현재값 카드(값·단위·출처·두 시각·상태·장애 재현)는 KRW/EUR/JPY/GBP 4개 통화 모두 정상 동작해.
- `data/history.json`에는 과거 KRW 기록 3건만 있고, 각 레코드에 `currency` 필드가 없어 (앱은 필드가 없으면 KRW로 취급). EUR/JPY/GBP를 선택하면 "날짜별 기록"과 "어제 대비 변화"에는 항상 "기록 없음"이 떠 — 의도된 상태야, 버그 아니야.
- `scripts/fetch-daily.mjs` (일별 자동 수집 스크립트, GitHub Actions에서 하루 1회 실행)는 아직 KRW 하나만 수집해.
- 개발용 부가 기능: URL에 `?currency=EUR` 처럼 붙이면 초기 통화를 디링크할 수 있어 (선택 사항, 지워도 됨).

### 3. 실행 명령

Node 18+ 필요(전역 `fetch` 사용), 외부 npm 패키지 의존성 없음. 저장소 루트에서 아무 정적 서버로 `index.html`을 서빙하면 돼. 예:

```
node -e "
const http=require('http'),fs=require('fs'),path=require('path');
http.createServer(async(req,res)=>{
  let p=req.url.split('?')[0]; if(p==='/')p='/index.html';
  try{const d=await fs.promises.readFile(path.join(process.cwd(),p));
    const ext=path.extname(p);
    res.writeHead(200,{ 'Content-Type':{'.html':'text/html','.css':'text/css','.js':'text/javascript','.j
    res.end(d);
```

```

}catch{res.writeHead(404);res.end('not found');}
}).listen(8766,()=>console.log('http://localhost:8766'));
"

```

날짜별 기록 수집만 테스트하려면: `node scripts/fetch-daily.mjs`

## 4. 통과 검사

아래 "고정 검사 10개"를 그대로 써야 해. 삭제·완화·기대값 변경 금지. 현재 결과: **8/10 PASS** (T-07, T-08만 FAIL). 목표는 10/10 PASS.

| ID   | 입력 (조작)                                                   | 관찰 가능한 기대값                                                                        |
|------|-----------------------------------------------------------|-----------------------------------------------------------------------------------|
| T-01 | 페이지를 처음 로드한다 (통화 선택을 건드리지 않음)                             | <code>#currency-select</code> 의 선택값이 "KRW" 이다                                     |
| T-02 | 통화 선택을 "EUR" 로 바꾼다                                        | <code>#unit</code> 의 텍스트가 "EUR / 1 USD" 로 바뀐다                                     |
| T-03 | 통화 선택을 "EUR" 로 바꾼다                                        | <code>#source-link</code> 의 href에 <code>symbols=EUR</code> 이 포함된다                 |
| T-04 | 통화 선택을 "JPY" 로 바꾼다                                        | 화면에 표시된 값이 실제 API 응답의 <code>rates.JPY</code> 값(소수 둘째 자리 반올림)과 일치한다                |
| T-05 | 통화 선택을 "GBP" 로 바꾼다 (정상 네트워크 상태)                           | <code>#status-badge</code> 의 텍스트가 "정상" 이다                                         |
| T-06 | URL에 <code>?fault=timeout</code> 을 붙이고 통화 선택을 "EUR" 로 바꾼다 | <code>#status-badge</code> 에 "오래된 데이터"와 "시간 초과"가 포함되고, 직전에 정상 조회된 값이 화면에 그대로 남아있다 |
| T-07 | 통화 선택을 "EUR" 로 바꾸고 "날짜별 기록"표를 확인한다                        | EUR로 기록된 날짜별 기록이 1건 이상 표시된다                                                       |
| T-08 | 통화 선택을 "EUR" 로 바꾸고 EUR 기록이 2건 이상일 때 "어제 대비 변화"를 확인한다      | 차이·방향·단위가 EUR / 1 USD 기준으로 표시된다                                                   |
| T-09 | 통화 선택을 "KRW" 로 되돌린다                                       | 기존 KRW 날짜별 기록 3건(2026-08-24, 2026-08-25, 2026-08-26)이 그대로 보존되어 있다                 |
| T-10 | <code>#currency-select</code> 의 옵션 목록을 확인한다               | 정확히 4개 옵션(KRW,EUR,JPY,GBP)이 있고 각 <code>value</code> 가 통화 코드와 같다                   |

## 5. 남은 문제

- `data/history.json`에 통화 구분을 추가해 (예: 각 레코드에 `currency` 필드 추가, 필드 없으면 KRW로 취급하는 기존 하위 호환 규칙은 유지).
- `scripts/fetch-daily.mjs`를 확장해서 KRW뿐 아니라 EUR/JPY/GBP도 하루 1회 수집·중복 방지 저장하도록 만들어 (날짜+통화 조합으로 dedupe).
- `.github/workflows/daily-fetch.yml`이 확장된 스크립트를 그대로 실행하는지 확인해.

4. `app.js` 의 `loadHistory()` / `renderCompare()` 는 이미 `currentCurrency` 로 필터링하도록 되어 있어서, 데이터만 채워지면 자동으로 통과할 가능성이 높아 — 먼저 코드를 고치기 전에 정말 그런지부터 확인해.
5. 실제 날짜 기록이 최소 1건이라도 쌓여야 T-07이 통과하고, 2건 이상이어야 T-08이 완전히 통과해. 하루 만에 여러 건을 몰아서 만들면 안 돼(과거 값을 조작하지 마) — 최소 1건이 실제로 수집되는 경로만 만들어 두면 충분해.

## 6. 다음 행동

---

1. 아래 붙여넣은 파일 내용을 기준으로 작업해.
2. "통과 검사" 10개를 먼저 (가능하면) 재현해서 정말 8/10인지 확인해.
3. "남은 문제" 1~3번 순서로 구현해.
4. 바뀐 파일은 **전체 내용**을 다시 보여줘 (diff만 말고, 복사해서 그대로 덮어쓸 수 있게).
5. 어떤 검사가 왜 통과/실패하는지 마지막에 요약해줘.

## 7. 금지 범위 (건드리지 말 것)

---

- 위 "고정 검사 10개"의 ID·입력·기대값을 삭제·완화·변경하지 마.
- 기존 KRW 날짜별 기록 3건(2026-08-24/25/26)의 값을 임의로 수정·삭제하지 마 (실제 수집값이야).
- 비밀키/토큰을 추가하지 마. Frankfurter API는 키가 필요 없고, 다른 통화도 같은 API로 전부 커버되니까 새 API/키가 필요 없어.

## 현재 파일 전체 내용

---

### index.html

```
<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>오늘의 진짜 정보판 - USD 환율</title>
<link rel="stylesheet" href="styles.css" />
</head>
<body>
  <main>
    <header>
      <h1>오늘의 진짜 정보판</h1>
      <p class="purpose">이 정보판은 <strong>미국 달러(USD) 대비 환율</strong>을 확인하기 위한 것이다 (통화
    </header>

    <section class="card" id="current-card">
      <h2>현재값</h2>
      <div class="current-value">
```

```

    <span id="value">불러오는 중...</span>
    <span id="unit" class="unit"></span>
</div>
<div class="currency-picker">
    <label for="currency-select">통화</label>
    <select id="currency-select">
        <option value="KRW">KRW</option>
        <option value="EUR">EUR</option>
        <option value="JPY">JPY</option>
        <option value="GBP">GBP</option>
    </select>
</div>
<dl class="meta">
    <dt>출처</dt>
    <dd><a id="source-link" href="#" target="_blank" rel="noopener">불러오는 중...</a></dd>
    <dt>출처 기준일 (원천 관측)</dt>
    <dd id="source-date">-</dd>
    <dt>조회 시각 (내 기기, Asia/Seoul)</dt>
    <dd id="fetched-at">-</dd>
    <dt>기준 시간대</dt>
    <dd>Asia/Seoul (KST, UTC+9)</dd>
    <dt>상태</dt>
    <dd id="status-badge" class="badge">-</dd>
</dl>
<button id="retry-btn" type="button">다시 시도</button>
</section>

<section class="card" id="fault-card">
    <h2>장애 재현 (검증용)</h2>
    <p class="hint">아래 링크를 누르면 각 장애 상태를 즉시 재현합니다. URL에 <code>?fault=</code> 쿼리로도
    <nav class="fault-links">
        <a href="#">정상</a>
        <a href="?fault=timeout">timeout</a>
        <a href="?fault=auth">인증 실패</a>
        <a href="?fault=ratelimit">호출 제한</a>
        <a href="?fault=offline">오프라인</a>
        <a href="?fault=format">응답 형식 변경</a>
    </nav>
</section>

<section class="card" id="history-card">
    <h2>날짜별 기록</h2>
    <p class="hint">기준 시간대: Asia/Seoul. 하루 1건만 저장됩니다.</p>
    <table id="history-table">
        <thead>
            <tr><th>날짜</th><th>값</th><th>단위</th><th>출처 기준일</th><th>조회 시각(KST)</th></tr>
        </thead>

```

```

        <tbody id="history-body">
          <tr><td colspan="5">불러오는 중...</td></tr>
        </tbody>
      </table>
    </section>

    <section class="card" id="compare-card">
      <h2>어제 대비 변화</h2>
      <div id="compare-result">비교할 기록을 불러오는 중...</div>
    </section>
  </main>
  <script src="app.js"></script>
</body>
</html>

```

## app.js

```

// 통화 선택 추가 (T05): USD 대비 KRW/EUR/JPY/GBP 중 골라 현재값·출처를 그 통화 기준으로 본다.
const CURRENCIES = {
  KRW: { unit: "KRW / 1 USD" },
  EUR: { unit: "EUR / 1 USD" },
  JPY: { unit: "JPY / 1 USD" },
  GBP: { unit: "GBP / 1 USD" },
};

function apiUrlFor(currency) {
  return `https://api.frankfurter.dev/v1/latest?base=USD&symbols=${currency}`;
}

function lastGoodKeyFor(currency) {
  return `t04-last-good-rate-${currency}`;
}

let currentCurrency = "KRW";

const TIMEZONE = "Asia/Seoul";
const TIMEOUT_MS = 8000;

const els = {
  value: document.getElementById("value"),
  unit: document.getElementById("unit"),
  sourceLink: document.getElementById("source-link"),
  fetchedAt: document.getElementById("fetched-at"),
  statusBadge: document.getElementById("status-badge"),
  sourceDate: document.getElementById("source-date"),
  retryBtn: document.getElementById("retry-btn"),
  historyBody: document.getElementById("history-body"),
  compareResult: document.getElementById("compare-result"),

```

```

currencySelect: document.getElementById("currency-select"),
};

function formatKST(isoString) {
  return new Intl.DateTimeFormat("ko-KR", {
    timeZone: TIMEZONE,
    year: "numeric", month: "2-digit", day: "2-digit",
    hour: "2-digit", minute: "2-digit", second: "2-digit",
    hour12: false,
  }).format(new Date(isoString));
}

function readLastGood(currency) {
  try {
    const raw = localStorage.getItem(lastGoodKeyFor(currency));
    return raw ? JSON.parse(raw) : null;
  } catch {
    return null;
  }
}

function writeLastGood(currency, data) {
  try {
    localStorage.setItem(lastGoodKeyFor(currency), JSON.stringify(data));
  } catch {
    // localStorage unavailable – page still works, just no cross-reload cache
  }
}

class FetchFault extends Error {
  constructor(type, message) {
    super(message);
    this.type = type;
  }
}

// 장애 5종을 결정적으로 재현하기 위한 시뮬레이션 계층.
// 실제 API에는 인증/요금제가 없어 auth.ratelimit은 모의로만 재현 가능하다.
async function fetchRate(faultMode, currency) {
  const apiUrl = apiUrlFor(currency);
  if (faultMode === "auth") {
    throw new FetchFault("auth", "인증 실패 (401 모의) – API 키/토큰이 거부되었습니다.");
  }
  if (faultMode === "ratelimit") {
    throw new FetchFault("ratelimit", "호출 제한 (429 모의) – 요청이 너무 많습니다.");
  }
  if (faultMode === "offline") {

```

```

    throw new FetchFault("offline", "오프라인 (모의) - 네트워크에 연결할 수 없습니다.");
  }
  if (faultMode === "timeout") {
    // 실제 응답 속도에 상관없이 시간 초과를 결정적으로 재현한다 (다른 모의 상태와 동일한 방식).
    await new Promise((r) => setTimeout(r, 300));
    throw new FetchFault("timeout", "시간 초과 (모의) - 응답이 지정 시간 안에 오지 않았습니다.");
  }

  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), TIMEOUT_MS);

  let res;
  try {
    res = await fetch(apiUrl, { signal: controller.signal });
  } catch (err) {
    clearTimeout(timer);
    if (err.name === "AbortError") {
      throw new FetchFault("timeout", "시간 초과 - 응답이 지정 시간 안에 오지 않았습니다.");
    }
    if (!navigator.onLine) {
      throw new FetchFault("offline", "오프라인 - 네트워크 연결이 없습니다.");
    }
    throw new FetchFault("offline", "네트워크 오류 - 요청을 보낼 수 없습니다.");
  }
  clearTimeout(timer);

  if (res.status === 401 || res.status === 403) {
    throw new FetchFault("auth", `인증 실패 (${res.status})`);
  }
  if (res.status === 429) {
    throw new FetchFault("ratelimit", "호출 제한 (429)");
  }
  if (!res.ok) {
    throw new FetchFault("format", `예상치 못한 응답 (HTTP ${res.status})`);
  }

  let data;
  try {
    data = await res.json();
  } catch {
    throw new FetchFault("format", "응답 형식이 예상과 다릅니다 (JSON 파싱 실패).");
  }

  const rate = data && data.rates && data.rates[currency];
  if (typeof rate !== "number" || faultMode === "format") {
    throw new FetchFault("format", `응답 형식이 예상과 다릅니다 (${currency} 항목 없음, 모의 포함).`);
  }

```

```

return {
  value: rate,
  unit: CURRENCIES[currency].unit,
  currency,
  source: apiUrl,
  sourceDate: data.date, // 출처(ECB) 자체가 매긴 환율 기준일 - 우리가 기록한 조회 시각과는 다른 시각
  fetchedAtISO: new Date().toISOString(),
};
}

function setBadge(kind, text) {
  els.statusBadge.textContent = text;
  els.statusBadge.className = "badge " + kind;
}

function renderGood(data) {
  els.value.textContent = data.value.toLocaleString("ko-KR", { maximumFractionDigits: 2 });
  els.unit.textContent = data.unit;
  els.sourceLink.href = data.source;
  els.sourceLink.textContent = data.source;
  els.sourceDate.textContent = data.sourceDate || "-";
  els.fetchedAt.textContent = formatKST(data.fetchedAtISO) + " (KST)";
  setBadge("ok", "정상");
}

function renderStale(lastGood, faultMessage) {
  if (lastGood) {
    els.value.textContent = lastGood.value.toLocaleString("ko-KR", { maximumFractionDigits: 2 });
    els.unit.textContent = lastGood.unit;
    els.sourceLink.href = lastGood.source;
    els.sourceLink.textContent = lastGood.source;
    els.sourceDate.textContent = lastGood.sourceDate || "-";
    els.fetchedAt.textContent = formatKST(lastGood.fetchedAtISO) + " (KST, 마지막 정상 조회)";
    setBadge("stale", `오래된 데이터 - ${faultMessage}`);
  } else {
    const fallbackUrl = apiUrlFor(currentCurrency);
    els.value.textContent = "값 없음";
    els.unit.textContent = "";
    els.sourceLink.href = fallbackUrl;
    els.sourceLink.textContent = fallbackUrl;
    els.sourceDate.textContent = "-";
    els.fetchedAt.textContent = "-";
    setBadge("error", faultMessage);
  }
}

```



```

async function load(bypassFault = false) {
  const params = new URLSearchParams(location.search);
  const faultMode = bypassFault ? null : params.get("fault");

  setBadge("stale", "불러오는 중...");
  try {
    const data = await fetchRate(faultMode, currentCurrency);
    writeLastGood(currentCurrency, data);
    renderGood(data);
  } catch (err) {
    const lastGood = readLastGood(currentCurrency);
    renderStale(lastGood, err.message || "알 수 없는 오류");
  }
}

// 재시도는 URL의 ?fault= 모의 상태를 무시하고 실제 호출을 다시 시도한다.
// (모의 장애 화면을 계속 보려면 링크로 재진입, 복구를 보려면 "다시 시도"를 누른다.)
els.retryBtn.addEventListener("click", () => load(true));

async function loadHistory() {
  let history = [];
  try {
    const res = await fetch("data/history.json", { cache: "no-store" });
    history = await res.json();
  } catch {
    els.historyBody.innerHTML = "<tr><td colspan='4'>기록을 불러오지 못했습니다.</td></tr>";
    els.compareResult.textContent = "기록을 불러오지 못해 비교할 수 없습니다.";
    return;
  }

  // 기존 기록에는 currency 필드가 없다 - 그런 기록은 KRW로 취급한다 (하위 호환).
  const filtered = history.filter((h) => (h.currency || "KRW") === currentCurrency);
  filtered.sort((a, b) => (a.date < b.date ? 1 : -1)); // 최신 날짜 먼저

  if (filtered.length === 0) {
    els.historyBody.innerHTML = "<tr><td colspan='5'>아직 저장된 날짜별 기록이 없습니다.</td></tr>";
  } else {
    els.historyBody.innerHTML = filtered
      .map(
        (h) =>
          `<tr><td>${h.date}</td><td>${h.rate.toLocaleString("ko-KR", { maximumFractionDigits: 2 })}</td>`
      )
      .join("");
  }

  renderCompare(filtered);
}

```

```

function renderCompare(historyDesc) {
  if (historyDesc.length < 2) {
    els.compareResult.textContent = "비교할 이전 기록이 아직 없습니다 (날짜별 기록 2건 이상 필요).";
    return;
  }
  const [latest, previous] = historyDesc;
  if (latest.unit !== previous.unit) {
    els.compareResult.textContent = "단위가 달라 비교값을 표시하지 않습니다.";
    return;
  }
  const diff = latest.rate - previous.rate;
  const direction = diff > 0 ? "상승 ▲" : diff < 0 ? "하락 ▼" : "변동 없음 -";
  const cls = diff > 0 ? "delta-up" : diff < 0 ? "delta-down" : "";
  els.compareResult.innerHTML =
    `${previous.date} (${previous.rate.toLocaleString("ko-KR")}) → ${latest.date} (${latest.rate.toLocaleString("ko-KR")})  

    <span class="${cls}">차이: ${diff >= 0 ? "+" : ""}${diff.toFixed(2)} ${latest.unit} (${direction})</span>`;
}

els.currencySelect.addEventListener("change", () => {
  currentCurrency = els.currencySelect.value;
  load(false);
  loadHistory();
});

// 자동 검증 편의용 (요구사항 밖 부가 기능): ?currency= 로 초기 통화를 디링크할 수 있다.
const urlCurrency = new URLSearchParams(location.search).get("currency");
if (urlCurrency && CURRENCIES[urlCurrency]) {
  els.currencySelect.value = urlCurrency;
}
currentCurrency = els.currencySelect.value; // T-01: 기본 선택값(KRW)에서 시작
load(false);
loadHistory();

```

## styles.css

```

:root {
  color-scheme: light dark;
  --bg: #ffffff;
  --fg: #1a1a1a;
  --muted: #666;
  --card-bg: #f6f7f9;
  --border: #e0e2e6;
  --ok: #1a7f37;
  --stale: #b45309;

```

```
--err: #c1121f;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: -apple-system, "Segoe UI", "Malgun Gothic", sans-serif;
  background: var(--bg);
  color: var(--fg);
  line-height: 1.5;
}

main {
  max-width: 640px;
  margin: 0 auto;
  padding: 24px 16px 64px;
}

header { margin-bottom: 24px; }
h1 { font-size: 1.4rem; margin-bottom: 4px; }
.purpose { color: var(--muted); }

.card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 20px;
  margin-bottom: 20px;
}
.card h2 { margin-top: 0; font-size: 1.05rem; }

.current-value { font-size: 2rem; font-weight: 700; margin: 8px 0 16px; }
.unit { font-size: 1rem; font-weight: 400; color: var(--muted); margin-left: 6px; }

.currency-picker { display: flex; align-items: center; gap: 8px; margin-bottom: 16px; }
.currency-picker label { color: var(--muted); font-size: 0.85rem; }
.currency-picker select {
  border: 1px solid var(--border); border-radius: 6px; padding: 4px 8px;
  background: var(--bg); color: var(--fg); font-size: 0.9rem;
}

.meta { display: grid; grid-template-columns: auto 1fr; gap: 4px 12px; margin: 0; }
.meta dt { color: var(--muted); font-size: 0.85rem; }
.meta dd { margin: 0 0 8px; }

.badge { display: inline-block; padding: 2px 10px; border-radius: 999px; font-size: 0.85rem; font-weight
```

```

.badge.ok { background: #d9f2df; color: var(--ok); }
.badge.stale { background: #fde9cd; color: var(--stale); }
.badge.error { background: #fbdadd; color: var(--err); }

button {
  background: var(--fg); color: var(--bg); border: none; border-radius: 8px;
  padding: 8px 16px; font-size: 0.9rem; cursor: pointer;
}
button:hover { opacity: 0.85; }

.hint { color: var(--muted); font-size: 0.85rem; }
.fault-links { display: flex; flex-wrap: wrap; gap: 8px; }
.fault-links a {
  border: 1px solid var(--border); border-radius: 8px; padding: 6px 10px;
  text-decoration: none; color: var(--fg); font-size: 0.85rem;
}

table { width: 100%; border-collapse: collapse; font-size: 0.9rem; }
th, td { text-align: left; padding: 6px 8px; border-bottom: 1px solid var(--border); }

#compare-result { font-size: 1.1rem; }
.delta-up { color: var(--ok); }
.delta-down { color: var(--err); }

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #14161a; --fg: #f0f1f3; --muted: #9aa0a8;
    --card-bg: #1d2025; --border: #2b2f36;
  }
  button { color: var(--bg); background: var(--fg); }
}

```

## data/history.json

```

[
  {
    "date": "2026-08-24",
    "rate": 1384.23,
    "unit": "KRW / 1 USD",
    "source": "https://api.frankfurter.dev/v1/latest?base=USD&symbols=KRW",
    "sourceDate": "2026-08-24",
    "fetchAtUtc": "2026-08-24T06:41:50.476Z",
    "timezone": "Asia/Seoul"
  },
  {

```

```

    "date": "2026-08-25",
    "rate": 1384.26,
    "unit": "KRW / 1 USD",
    "source": "https://api.frankfurter.dev/v1/latest?base=USD&symbols=KRW",
    "fetchAtUtc": "2026-08-25T01:47:09.060Z",
    "timezone": "Asia/Seoul"
  },
  {
    "date": "2026-08-26",
    "rate": 1383.07,
    "unit": "KRW / 1 USD",
    "source": "https://api.frankfurter.dev/v1/latest?base=USD&symbols=KRW",
    "sourceDate": "2026-08-25",
    "fetchAtUtc": "2026-08-26T01:32:15.981Z",
    "timezone": "Asia/Seoul"
  }
]

```

## scripts/fetch-daily.mjs

```

// 하루 1회, 기준 시간대(Asia/Seoul) 날짜로 환율 스냅샷 1건을 data/history.json에 추가한다.
// 같은 날짜가 이미 있으면 아무것도 하지 않는다 (중복 방지).
import { readFile, writeFile } from "node:fs/promises";

const API_URL = "https://api.frankfurter.dev/v1/latest?base=USD&symbols=KRW";
const TIMEZONE = "Asia/Seoul";
const HISTORY_PATH = new URL("../data/history.json", import.meta.url);

function todayInTimezone(tz) {
  return new Intl.DateTimeFormat("en-CA", { timeZone: tz }).format(new Date()); // YYYY-MM-DD
}

async function main() {
  const res = await fetch(API_URL);
  if (!res.ok) {
    throw new Error(`API 호출 실패: HTTP ${res.status}`);
  }
  const data = await res.json();
  const rate = data?.rates?.KRW;
  if (typeof rate !== "number") {
    throw new Error("응답에 KRW 항목이 없습니다.");
  }

  const raw = await readFile(HISTORY_PATH, "utf-8").catch(() => "[]");
  const history = JSON.parse(raw);

```

```

const date = todayInTimezone(TIMEZONE);
if (history.some((h) => h.date === date)) {
  console.log(`[skip] ${date} 기록이 이미 있습니다.`);
  return;
}

history.push({
  date,
  rate,
  unit: "KRW / 1 USD",
  source: API_URL,
  sourceDate: data.date, // 출처(ECB)가 매긴 환율 기준일 - 우리 조회 시각과 별개
  fetchedAtUtc: new Date().toISOString(),
  timezone: TIMEZONE,
});
history.sort((a, b) => (a.date < b.date ? -1 : 1));

await writeFile(HISTORY_PATH, JSON.stringify(history, null, 2) + "\n", "utf-8");
console.log(`[added] ${date} = ${rate}`);
}

main().catch((err) => {
  console.error(err);
  process.exit(1);
});

```

## **.github/workflows/daily-fetch.yml**

```

name: Daily exchange rate snapshot

on:
  schedule:
    # 매일 00:10 UTC = 09:10 KST
    - cron: "10 0 * * *"
  workflow_dispatch: {}

permissions:
  contents: write

jobs:
  fetch:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:

```

```
node-version: "20"
- run: node scripts/fetch-daily.mjs
- name: Commit if changed
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    if ! git diff --quiet -- data/history.json; then
      git add data/history.json
      git commit -m "chore: daily exchange rate snapshot"
      git push
    else
      echo "No change."
    fi
```

---붙여넣을 내용 끝---